

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Assessment Tool Weightage: 30%

QUESTIONS:

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

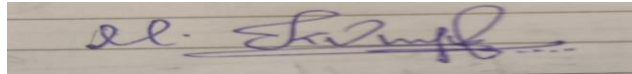
Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes.
- Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

Code of Conduct:

I KUMARAN M(192525022) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.

Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.
Clarity	10	Clear, well-organized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.

ANSWERS

1. Design an Autoencoder-Based Feature Learning System

Problem:

A manufacturing company has **high-dimensional sensor data** from machines and wants to detect defective products. Processing all sensor features directly may be computationally expensive and may contain redundant information.

Requirements:

- Reduce the number of input features.
- Learn important hidden patterns automatically.
- Reconstruct the original sensor data.
- Provide compact features for defect detection.

Proposed Model: Undercomplete Autoencoder

An **undercomplete autoencoder** has a latent representation with fewer dimensions than the input.

Architecture:

High-Dimensional Sensor Input



Input Layer (100)



Dense Layer (64)

ReLU Activation



Dense Layer (32)

ReLU Activation

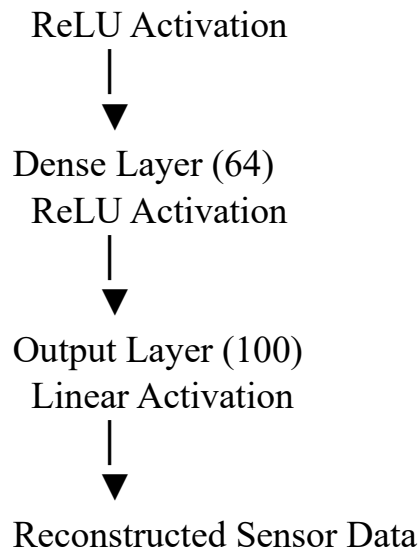


Latent Representation (10)

Linear / ReLU



Dense Layer (32)



Encoder:

The encoder converts the original high-dimensional input into a smaller representation.

For example:

100 sensor features $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 10 latent features

The latent vector contains the most important information needed to reconstruct the input.

Decoder:

The decoder reconstructs the original sensor data from the latent representation:

10 $\rightarrow$ 32 $\rightarrow$ 64 $\rightarrow$ 100 features

Activation Functions:

- **ReLU** is used in hidden layers because it provides nonlinear feature learning.
- **Linear activation** can be used at the output because sensor values can have a continuous range.

Reconstruction Loss:

Mean Squared Error (MSE) can be used:

$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

where:

- (x_i) = original sensor value
- $(\hat{x}_i)$ = reconstructed value

How Dimensionality Reduction Occurs

The bottleneck layer contains only **10 features instead of 100**. The network is forced to learn the most important information rather than simply copying the input.

Feature Learning:

The latent representation may automatically capture patterns related to:

- Temperature
- Pressure
- Vibration
- Speed
- Electrical measurements
- Machine operating conditions

These learned features can then be given to a classifier for defect detection.

Training Process:

1. Input sensor data into the encoder.
2. Compress it into the latent vector.
3. Reconstruct the input using the decoder.
4. Calculate reconstruction error.
5. Backpropagate the error.
6. Update weights using an optimizer such as Adam.
7. Repeat for multiple epochs.

Evaluation:

Suitable measures include:

- Reconstruction MSE
- MAE
- Classification accuracy
- Precision
- Recall
- F1-score

Expected Outcome:

The model produces a **compact and informative representation** of sensor data while removing redundant information. This improves the efficiency of downstream defect detection.

2. Design a Robust Regularized Autoencoder

Problem:

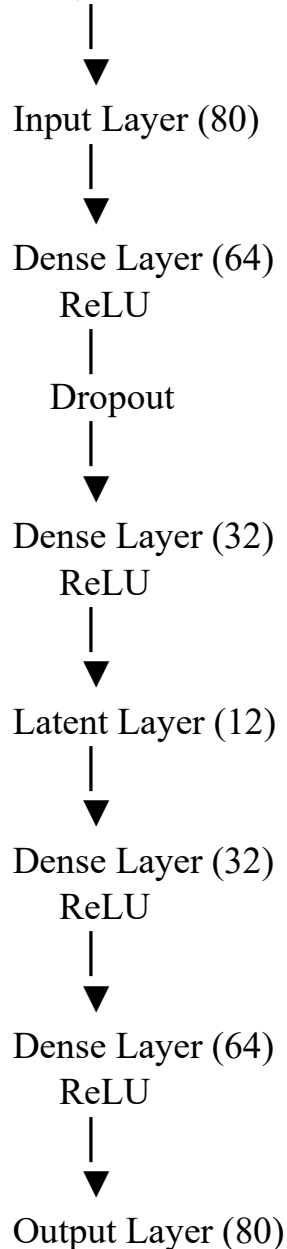
Healthcare datasets may contain noisy, redundant, and high-dimensional patient-record features. A normal autoencoder may learn unnecessary patterns from the training data. A **regularized autoencoder** can produce more robust representations and reduce overfitting.

Proposed Model:

A **Denoising Autoencoder with L2 regularization and dropout** can be used.

Architecture:

Noisy Patient Features



Linear



Reconstructed Features

Regularization Strategy:

Use three mechanisms:

1. Dropout

Random neurons are temporarily removed during training. This prevents the model from depending too strongly on specific neurons.

2. L2 Regularization

A penalty is added to large network weights:

$$[L_{\text{total}} = L_{\text{reconstruction}} + \lambda \sum W^2]$$

This encourages simpler model parameters.

3. Input Noise

Noise can be deliberately added to the input:

Original Patient Data



Add Controlled Noise



Denoising Autoencoder



Clean Reconstruction

The model therefore learns important underlying patterns rather than memorizing noisy values.

Training Objective:

The complete objective can be written as:

$$[L = L_{\text{MSE}} + \lambda \|W\|_2^2]$$

where (L_{MSE}) measures reconstruction quality and the second term provides regularization.

Why It Reduces Overfitting

Without regularization, the autoencoder may simply memorize the training samples.

Regularization:

- Limits excessive weight values.
- Prevents excessive dependence on individual neurons.
- Encourages general features.
- Improves performance on unseen data.

Useful Feature Extraction

The latent layer may capture meaningful relationships among patient-record features.

For example:

- Vital-sign patterns
- Laboratory measurements
- Patient history patterns
- General risk-related feature combinations

These representations can be passed to a downstream classification model.

Evaluation:

- Reconstruction MSE
- Classification accuracy
- Precision
- Recall
- F1-score
- Validation loss

Expected Outcome:

The proposed regularized autoencoder produces **stable and useful patient representations** while reducing overfitting and the effect of noise.

3. Design a Deep Generative Model

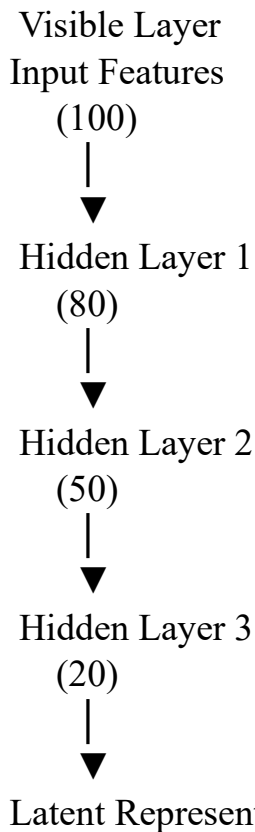
Problem:

A recommendation or content-generation system needs to learn complex probability distributions from existing data.

A **Deep Belief Network (DBN)** can learn hierarchical representations and generate new samples similar to the training data.

Proposed Model: Deep Belief Network:

Architecture:



A DBN is composed of multiple hidden layers, commonly built using **Restricted**

Boltzmann Machines (RBMs):

Visible Layer:

The visible layer receives the original data.

For a recommendation system, input could represent:

- User preferences
- Item ratings
- Purchase history
- Content features

Hidden Layers:

Hidden layers learn increasingly complex patterns.

For example:

User Preferences



Basic Preference Patterns



Complex User Interests



Latent User Representation

Learning Process:

DBN training is generally performed in two stages.

Stage 1: Layer-wise Pre-training

Each RBM is trained individually.

Visible → Hidden 1

Hidden 1 → Hidden 2

Hidden 2 → Hidden 3

This allows each layer to learn useful representations.

Stage 2: Fine-tuning:

After pre-training, the complete network can be fine-tuned to improve the final objective.

RBM Concept:

An RBM contains:

- Visible units
- Hidden units

There are connections between visible and hidden layers, but not normally between units within the same layer.

Visible: ● ● ● ● ●



Hidden: ● ● ● ● ●

Why DBN?

A DBN is suitable because:

- It learns hierarchical features.
- It can model complex data distributions.
- It provides useful latent representations.
- It can be used for generative tasks.

- Layer-wise training can help when learning complex representations.

Evaluation:

Depending on the application:

- Reconstruction error
- Recommendation accuracy
- Precision@K
- Recall@K
- Generated-sample quality
- User preference similarity

Expected Outcome:

The DBN learns hidden patterns in the training data and can generate or recommend content based on the learned probability distribution.

4. Design a Stochastic and Contractive Representation Model

Problem:

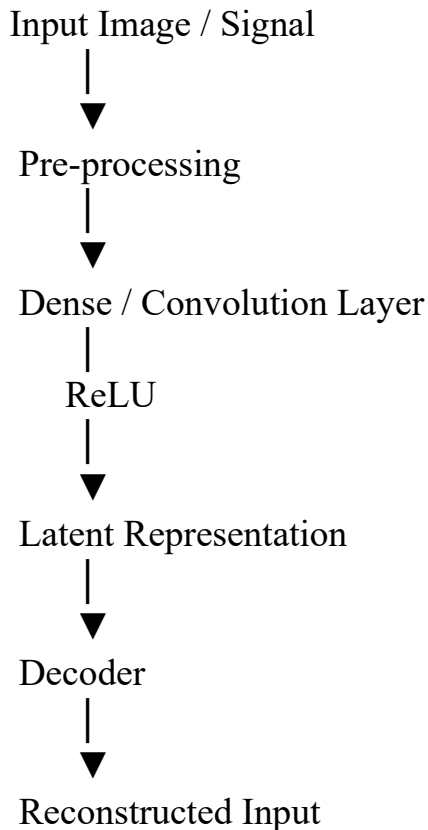
In image and signal-processing applications, small changes in the input should not cause large changes in the learned representation.

For example, an image may experience:

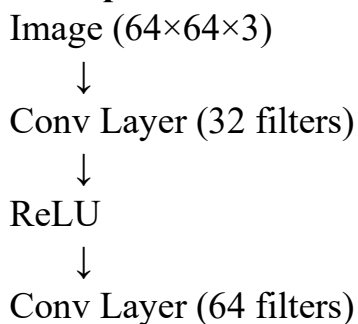
- Small rotation
- Sensor noise
- Brightness variation
- Small signal disturbances

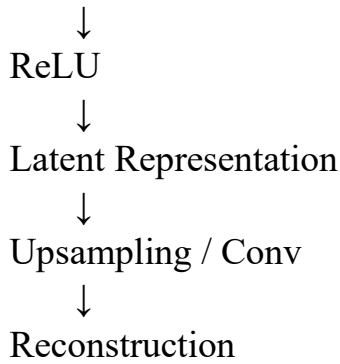
A **Contractive Autoencoder (CAE)** can be designed to learn stable representations.

Proposed Architecture:



Example Architecture for Images





Contractive Mechanism:

The model minimizes reconstruction error while also penalizing sensitivity of the encoder to small changes in the input.

The objective can be written as:

$$L = L_{\text{reconstruction}} + \lambda \|J_f(x)\|_F^2$$

where:

- $(L_{\text{reconstruction}})$ = reconstruction error
- $(J_f(x))$ = Jacobian matrix of the encoder
- (λ) = regularization strength
- $(\|J_f(x)\|_F^2)$ = Frobenius norm of the Jacobian

Meaning of the Contractive Penalty:

The penalty encourages the encoder output to change only slightly when the input changes slightly.

Therefore:

Small Input Change



Small Representation Change



Stable Features

Example:

Suppose two images contain the same object but have slightly different brightness.

A normal encoder might produce noticeably different representations.

A contractive encoder attempts to produce:

Image A $\rightarrow$ Feature Vector A

Image B $\rightarrow$ Feature Vector $\approx$ A

Advantages:

- Robust to small perturbations.
- Learns stable features.
- Reduces sensitivity to noise.
- Useful for image recognition.
- Useful for signal processing.
- Improves downstream classification.

Evaluation:

- Reconstruction error
- Classification accuracy
- Robustness under noise
- Representation similarity
- Performance under small input perturbations

Expected Outcome:

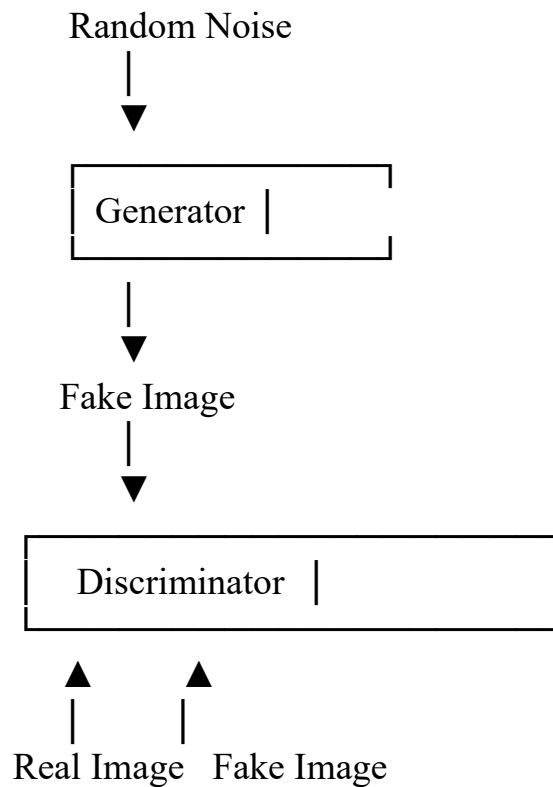
The contractive model learns representations that preserve important information while being relatively insensitive to small changes in the input.

5. Design a GAN-Based Data Generation System

Problem:

An organization has a limited number of training images. A **Generative Adversarial Network (GAN)** can generate synthetic images that resemble the original training data.

GAN Architecture



1. Generator

The Generator receives a random noise vector:

[
 $z \sim N(0,1)$
]

and converts it into a synthetic image.

Example:

Random Vector

↓
Dense Layer
↓
Upsampling
↓
Convolution
↓
Generated Image

For image generation, a **DCGAN-style architecture** can use convolutional and transposed-convolutional layers.

Generator Output:

The output is an image designed to resemble images from the training dataset.

2. Discriminator

The Discriminator receives either a real image or a generated image.

Input Image



Convolution Layers



Feature Extraction



Dense Layer



Probability



Real / Fake

The output indicates whether the image is likely to be real or generated.

Adversarial Training:

The Generator and Discriminator compete with each other.

Generator Goal:

Generate images that the Discriminator classifies as real.

Discriminator Goal:

Correctly identify:

- Real training images
- Fake generated images

The classical GAN objective is:

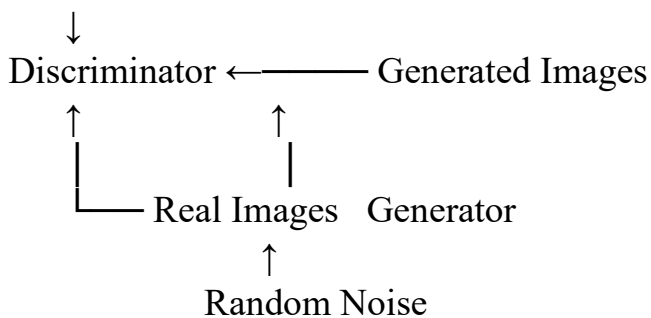
$$\begin{aligned} & [\\ & \min_{\mathbf{G}} \max_{\mathbf{D}} V(\mathbf{D}, \mathbf{G}) \\ & E_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] \\ & + \\ & E_{\mathbf{z} \sim p_{\mathbf{z}}} [\log(1 - D(\mathbf{G}(\mathbf{z})))] \\ &] \end{aligned}$$

Training Process:

1. Select real images from the dataset.
2. Generate fake images using random noise.
3. Train the Discriminator using real and fake images.
4. Train the Generator to fool the Discriminator.
5. Repeat this process iteratively.
6. As training progresses, the Generator attempts to produce increasingly realistic samples.

Workflow:

Training Images



Evaluation of Generated Images:

Several methods can be used:

1. Visual Inspection

Check whether generated images appear realistic.

2. Diversity

Ensure the Generator does not produce nearly identical images.

3. FID

Fréchet Inception Distance (FID) can be used to compare the distribution of generated images with real images.

4. Downstream Accuracy

Train a classifier using real + synthetic images and compare its performance with a classifier trained only on real data.

GAN vs Autoencoder:

Feature	Autoencoder	GAN
Main purpose	Representation/reconstruction	Data generation
Structure	Encoder + Decoder	Generator + Discriminator
Training	Reconstruction loss	Adversarial training
Output quality	Can be blurry	Often sharper/more realistic
Latent representation	Explicit	Implicit/random latent space
Main application	Feature learning	Synthetic data generation
Data augmentation	Possible	Highly suitable

Why GAN is Suitable

For the given problem, the organization specifically wants **realistic synthetic images**. GANs are designed to learn the distribution of real images and generate new samples from that distribution.

They can:

- Increase dataset size.
- Improve training-data diversity.
- Generate realistic samples.
- Help reduce the problem of limited training data.

Expected Outcome:

The trained Generator produces synthetic images that resemble the real dataset. These images can be used as additional training data, provided their quality and diversity are carefully evaluated.

Overall Design Summary:

Question	Model	Main Purpose	Key Technique
Q1	Undercomplete Autoencoder	Dimensionality reduction	Bottleneck latent layer
Q2	Regularized Autoencoder	Robust feature learning	Dropout + L2 + denoising
Q3	DBN	Generative representation	Layer-wise RBM learning
Q4	Contractive Autoencoder	Stable representation	Jacobian penalty
Q5	GAN	Synthetic data generation	Generator vs Discriminator